

Documentation Notes

1. Main idea

Odoo can be extended internally using modules, but its data can also be accessed externally through APIs.

This means other systems can connect to Odoo, such as:

```
Webflow  
Shopify  
FlutterFlow  
Make / n8n  
Python scripts  
Postman  
Payment gateways
```

For Odoo 19, the important new API is:

```
External JSON-2 API
```

It uses this endpoint:

```
/json/2
```

2. External JSON-2 API

What is JSON-2 API?

The **JSON-2 API** is Odoo 19's newer external API style.

It lets external systems call Odoo models and methods over HTTP.

Simple meaning:

JSON-2 lets us send JSON requests to Odoo to read, create, update, or delete data.

Example:

```
/json/2/res.partner/search_read
```

This means:

Part	Meaning
<code>/json/2</code>	Odoo JSON-2 API endpoint
<code>res.partner</code>	Odoo model for contacts/customers
<code>search_read</code>	Method/action to execute

3. Request structure

Odoo JSON-2 requests are sent as:

```
POST /json/2/<model>/<method>
```

Example:

```
POST /json/2/res.partner/search_read
```

Request parts

Part	Meaning
URL	Tells Odoo which model and method to call
Headers	Authentication and request information
Body	JSON data/instructions sent to Odoo

4. Important HTTP headers

Header	Required?	Meaning
<code>Authorization</code>	Required	Sends the API key as bearer token
<code>Content-Type</code>	Required	Tells Odoo the body is JSON
<code>X-Odoo-Database</code>	Sometimes required	Tells Odoo which database to use
<code>User-Agent</code>	Recommended	Identifies the software/tool making the request
<code>Host</code>	Required by HTTP	Usually handled automatically

Example headers:

```
Authorization: bearer YOUR_API_KEY
Content-Type: application/json; charset=utf-8
X-Odoo-Database: edu-test4620
User-Agent: Nexulon Week 1 API Test
```

Important:

In real files, never write the real API key. Use variables or environment variables.

5. Request body

The body is a JSON object.

Common body fields:

Field	Meaning
<code>ids</code>	Record IDs to apply the method to
<code>context</code>	Extra settings, such as language
<code>domain</code>	Filter/search condition
<code>fields</code>	Fields we want returned
<code>vals</code>	Values to update
<code>vals_list</code>	Values to create

Example:

```
{
  "context": {
    "lang": "en_US"
  },
  "domain": [],
  "fields": ["id", "name", "email"],
  "limit": 10
}
```

6. Response structure

If the request works, Odoo returns:

```
200 OK
```

with the result in the response body.

Example:

```
[
  {
    "id": 25,
    "name": "Deco Addict"
  }
]
```

If there is an error, Odoo returns a `4xx` or `5xx` status code with an error object.

Example error:

```
401 Unauthorized
```

Usually means:

Invalid or missing API key

7. API keys

How to create an API key

In Odoo:

```
Preferences
↓
Account Security
↓
New API Key
```

You need:

```
Description
Duration
```

The API key is shown only once, so it must be copied and stored securely.

Important security notes:

```
Do not share API keys
Do not put real keys in screenshots
Do not commit keys to GitHub
Revoke exposed keys immediately
Use short duration keys for testing
Rotate long-term keys regularly
```

Odoo recommends short durations for interactive/testing use.

8. API key rotation

API keys should be rotated regularly.

Safe rotation process:

1. Generate a new API key
2. Store the new key securely
3. Update services to use the new key
4. Test that the new key works
5. Revoke the old key

Best practice:

| Use a separate API key for each service or integration.

Example:

```
One key for Webflow integration
One key for Shopify integration
One key for FlutterFlow integration
```

This makes it easier to revoke one integration without affecting all others.

9. Access rights

The JSON-2 API follows Odoo's normal security rules.

This means the API key can only do what the Odoo user is allowed to do.

Odoo checks:

```
Access rights
Record rules
Field access
User permissions
```

For testing, using a personal/admin account is acceptable.

For real integrations, use:

```
Dedicated bot user / integration user
```

Why?

```
Limits permissions  
Improves security  
Makes logs clearer  
Avoids using personal admin accounts
```

Nexulon note:

For production, each integration should use a dedicated Odoo integration user with only the permissions it needs.

10. Database header

Sometimes Odoo needs the database name in the request.

Header:

```
X-Odoo-Database: edu-test4620
```

This is important when one Odoo server hosts multiple databases.

For our test database, we used:

```
X-Odoo-Database: edu-test4620
```

11. Transactions

Each JSON-2 API call runs in its own database transaction.

Simple meaning:

Each API request is treated as one separate operation.

If the request succeeds:

```
Odoo commits/saves the change
```

If the request fails:

```
Odoo rolls it back / discards it
```

Important note:

| You cannot chain multiple JSON-2 calls inside one single transaction.

So for sensitive operations like:

```
Payments  
Reservations  
Stock changes  
Order confirmation
```

it is better to call one Odoo method that handles the full operation safely.

Example:

```
sale.order/action_confirm
```

Instead of doing many separate risky steps manually.

12. Important Odoo model and methods

For Week 1, we are using:

```
res.partner
```

This is the Odoo model for:

Contacts
Customers
Companies
Vendors

Methods we need

Method	Meaning	CRUD
<code>search_read</code>	Search and read records	Read
<code>create</code>	Create a new record	Create
<code>write</code>	Update existing record	Update
<code>unlink</code>	Delete record	Delete

13. Example: Read contacts

URL:

```
POST /json/2/res.partner/search_read
```

Body:

```
{
  "context": {
    "lang": "en_US"
  },
  "domain": [],
  "fields": ["id", "name", "email", "phone", "city"],
  "limit": 10
}
```

Meaning:

Read up to 10 contacts and return selected fields.

14. Example: Create contact

URL:

```
POST /json/2/res.partner/create
```

Body:

```
{
  "context": {
    "lang": "en_US"
  },
  "vals_list": {
    "name": "API Test Contact - Enjy",
    "email": "api.test.enjy@nexulon.com",
    "phone": "+201000000000",
    "city": "Cairo"
  }
}
```

Meaning:

| Create a new contact in Odoo.

15. Example: Update contact

URL:

```
POST /json/2/res.partner/write
```

Body:

```
{
  "ids": [8],
}
```

```
"context": {  
  "lang": "en_US"  
},  
"vals": {  
  "phone": "+201000000001",  
  "city": "Alex"  
}  
}
```

Meaning:

| Update contact ID 8 with a new phone and city.

16. Example: Delete contact

URL:

```
POST /json/2/res.partner/unlink
```

Body:

```
{  
  "ids": [8],  
  "context": {  
    "lang": "en_US"  
  }  
}
```

Meaning:

| Delete contact ID 8.

17. XML-RPC / JSON-RPC vs JSON-2

Odoo also has older APIs:

```
XML-RPC
JSON-RPC
```

But in Odoo 19, Odoo says these older APIs are deprecated and scheduled for removal in the future.

The replacement is:

```
External JSON-2 API
```

Simple comparison

API type	Status	Main idea
XML-RPC	Old / deprecated	Uses XML-RPC endpoint and login/password or API key as password
JSON-RPC	Old / deprecated	Uses JSON-RPC format
JSON-2	New in Odoo 19	Uses HTTP POST, JSON body, bearer API key

18. How authentication changed

Old XML-RPC / JSON-RPC

Needed:

```
Database
Username
Password/API key
User ID
Model
Method
Arguments
```

New JSON-2

Needs:

```
URL with model and method  
Authorization bearer API key  
JSON body  
Optional database header
```

Example:

```
Authorization: bearer API_KEY
```

This is simpler and closer to modern API style.

19. External RPC API notes

The older External RPC API uses:

```
/xmlrpc/2/common  
/xmlrpc/2/object  
/jsonrpc
```

Important concepts:

Method	Meaning
<code>search</code>	Find record IDs
<code>read</code>	Read record data
<code>search_read</code>	Search + read in one call
<code>create</code>	Create record
<code>write</code>	Update record
<code>unlink</code>	Delete record
<code>fields_get</code>	Inspect model fields

For our Week 1 work, we do **not** need to use XML-RPC. We only need to understand that JSON-2 is the newer replacement.

20. Extract API

The **Extract API** is different from the normal Odoo external API.

It is used for document processing with OCR and AI.

It can process:

```
Invoices
Bank statements
Expenses
Resumes
```

It extracts fields such as:

```
Total
Due date
Invoice lines
Balances
Name
Email
Phone
```

Important:

| This is not needed for our current Week 1 deliverable.

It may be useful later for clients who need document digitization or accounting automation.

21. Webhooks note

The documentation also mentions webhook-style behavior in the Extract API.

A webhook means:

Instead of checking repeatedly if something is ready, the system sends a POST request to a URL when the result/event is ready.

Simple difference:

Method	Meaning
Polling	Keep checking repeatedly
Webhook	Get notified automatically when something happens

Nexulon example:

```
Webflow form submitted
↓
Webhook sends data to Make/n8n
↓
Make/n8n sends data to Odoo CRM
```

This is why webhooks are important for integrations.

22. What matters for Week 1 deliverable

For Week 1, focus on these only:

```
JSON-2 API
API keys
Authorization header
X-Odoo-Database header
res.partner model
search_read
create
write
unlink
Postman testing
Python script
Webhook vs polling concept
```

Do not spend too much time now on:

```
Programmatic API key management
Extract API
```

```
Dynamic documentation
Database service
Custom models through RPC
Deep transactions
```

These can be learned later.

23. One-page summary

Odoo JSON-2 API

```
POST /json/2/<model>/<method>
```

Example:

```
POST /json/2/res.partner/search_read
```

Required headers

```
Authorization: bearer API_KEY
Content-Type: application/json; charset=utf-8
X-Odoo-Database: DATABASE_NAME
```

Main model

```
res.partner = Contacts / Customers
```

Main methods

```
search_read = read
create = create
```



```
write = update  
unlink = delete
```

Security

```
Use API keys carefully  
Do not expose keys  
Use dedicated integration users for production  
Rotate keys regularly
```

Webhook idea

```
Webhook = external system pushes data automatically when an event happens  
Polling = system keeps checking repeatedly
```

One sentence to remember

Odoo JSON-2 API lets external systems securely call Odoo models and methods through HTTP using a bearer API key, so tools like Webflow, Shopify, FlutterFlow, Make, and Python scripts can read, create, update, or delete Odoo records.